

The Software requirements document

⇒ The software requirements document.

- **Purpose:** This document tells the developers what the system needs to do. It's a formal way to say what the system should achieve but not how to build it.

- **Contents:** It includes both user requirements (what user need) and system requirements (specific details about what the system must do).

- **Nature:** It's not about design or implementation but focuses on functionality and requirements.

⇒ Agile methods and requirements:

- **Agile approach:** Agile methods, such as Extreme Programming (XP), often avoid detailed requirements documents because requirements change rapidly.

- **User stories:** Instead of long documents, they use short, simple descriptions of features called "user stories".

- **Applicability:** This approach is good for business systems where needs can be discussed and changed quickly, but it's harder for complex systems needing detailed upfront analysis.

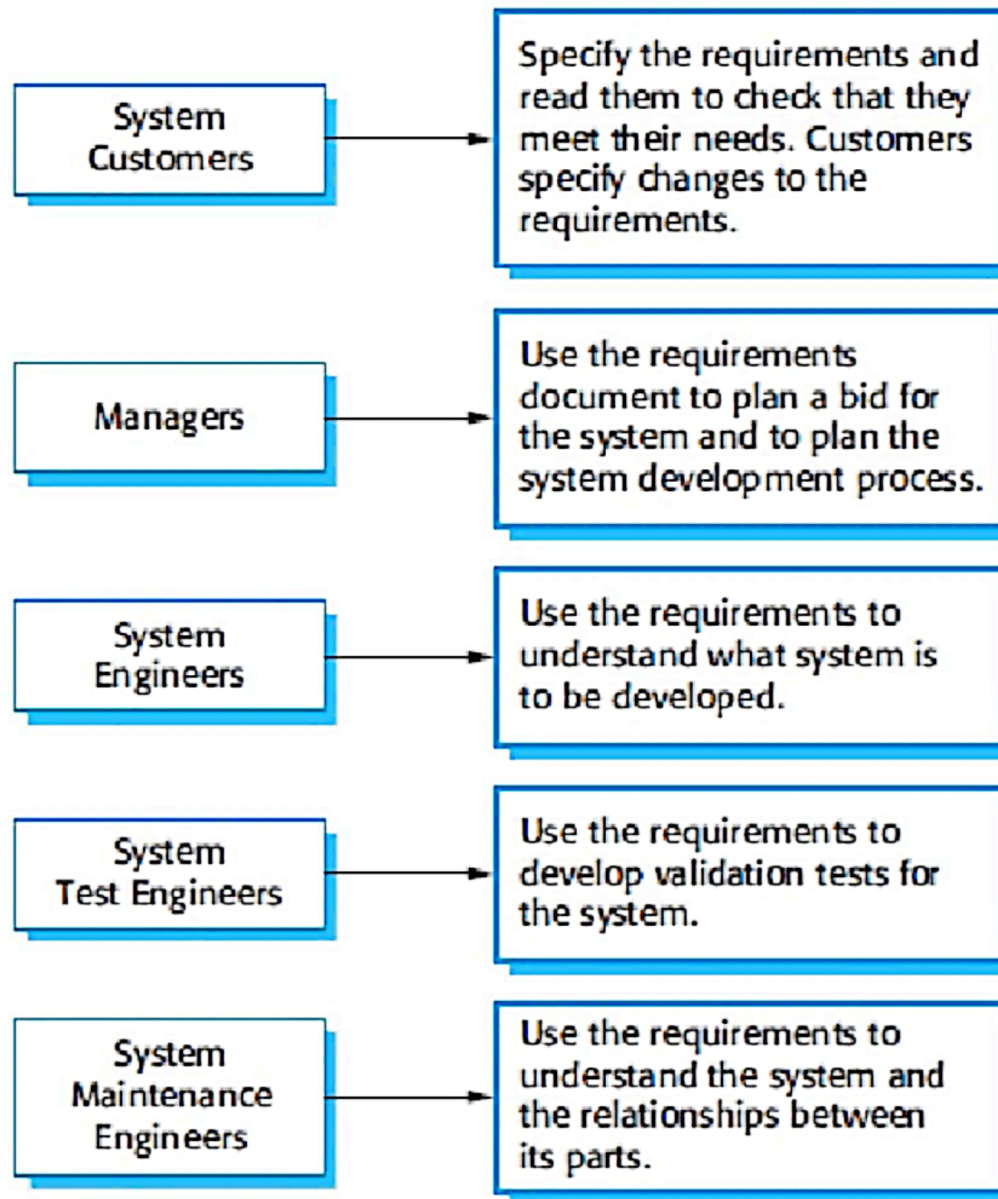
⇒ Requirements specification:

- **User requirements:** These are written in simple language for users and customers who may not be technical.
- **System requirements:** These are detailed and technical, specifying what the system must do.
- **Completeness:** Both types should be as complete and detailed as possible to form a clear contract for what the system will deliver.

⇒ Ways of writing a system requirements specification:



Users of a requirements document





Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Graphical notations	Graphical models, enhanced by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract



Example requirements for the insulin pump software system

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (*Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.*)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (*A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.*)



Guidelines for writing requirements

- ❑ **Invent a standard format** and use it for all requirements.(single sentence!!)
- ❑ **Use language in a consistent way.** Use shall for mandatory requirements, should for desirable requirements.
- ❑ **Use text highlighting** to identify key parts of the requirement.
- ❑ **Avoid** the use of **computer jargon** (specialised words).
- ❑ **Include an explanation** (rationale) of why a requirement is necessary.

- **Standard format:** Use a consistent format for all the requirements.

- **Clear language:** Use plain and clear language, avoiding technical jargon.

- **Mandatory vs desirable:** Use "shall" for must-have requirements and "should" for nice-to-have requirements.

- **Highlighting:** Highlight key parts to make them stand out.

- **Rationale:** Include explanations for why each requirement is necessary.

⇒ Problems with natural language:

- **Lack of clarity:**

Natural language can be vague and lead to misunderstandings.

- **Requirements confusion:**

Functional, non-functional... requirements and design information tend to be mixed up.

- **Requirements incorporation:**

Several different requirements may

- be expressed together in a single requirement.

⇒ Structured specification:

- **Approach:** Limits how requirements are written to ensure clarity and consistency.
- **Usage:** works well for technical systems (like embedded control systems) but might be too rigid for business requirements.

⇒ Form-based specifications:

- **Components:** These specify the function, inputs, outputs, actions, conditions; and any side effects.
- **Purpose:** Provides a clear and structured way to describe system functions.

⇒ Tabular specifications:

- **Usage:** uses tables to describe different scenarios and actions.
- **Example:** For an insulin pump, tables might show how to calculate insulin doses based on varying blood sugar levels.



A structured specification of a requirement for an insulin pump

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: safe sugar level.

Description

Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2); the previous two readings (r0 and r1).

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose—the dose in insulin to be delivered.

Destination Main control loop.



A structured specification of a requirement for an insulin pump

Action

CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requirements

Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition

The insulin reservoir contains at least the maximum allowed single dose of insulin.

Post-condition r0 is replaced by r1 then r1 is replaced by r2.

Side effects None.



Tabular specification of computation for an insulin pump

Condition	Action
Sugar level falling ($r2 < r1$)	<u>CompDose</u> = 0
Sugar level stable ($r2 = r1$)	<u>CompDose</u> = 0
Sugar level increasing and rate of increase decreasing ($(r2 - r1) < (r1 - r0)$)	<u>CompDose</u> = 0
Sugar level increasing and rate of increase stable or increasing ($(r2 - r1) \geq (r1 - r0)$)	<u>CompDose</u> = round $((r2 - r1)/4)$ If rounded result = 0 then <u>CompDose</u> = <u>MinimumDose</u>

⇒ Requirements engineering processes:

- **Steps:** Involves elicitation (gathering requirements), analysis (understanding requirements), validation (checking requirements), and management (handling changes).

- **Iterative nature:** These steps are repeated as needed to refine and clarify requirements.

⇒ Requirements elicitation and analysis:

- **Stakeholders:** Involves users, managers, domain experts etc who help define what the system should do.

- **Activities:** Discovering, organizing, prioritizing, and documenting requirements through collaboration with stakeholders.

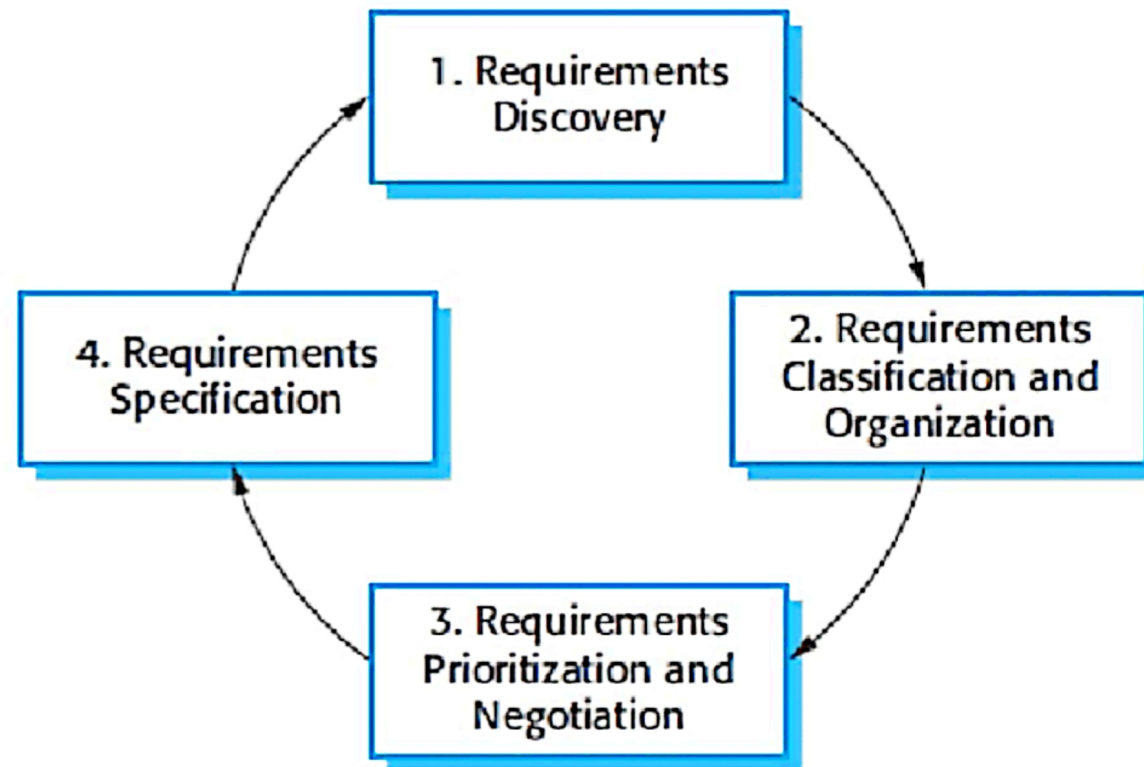
⇒ Problems of requirements analysis:

- **Unclear needs:** Stakeholders might not know exactly what they want or express it in their own terms.

- **Conflicts:** Different stakeholders may have conflicting requirements.



Requirements elicitation and analysis process activities





Interviewing

- Formal or informal interviews with stakeholders are part of most RE processes.
- Types of interview
 - **Closed interviews** based on pre-determined list of questions
 - **Open interviews** where there is no predefined agenda. Various issues are explored with stakeholders.
- Effective interviewing
 - **Be open-minded**, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.
 - Prompt the interviewee to get discussions going using a requirements proposal, or by working together on a prototype system.



Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
 - Requirements engineers cannot understand specific domain terminology; It is impossible for stakeholders to discuss domain requirements without using this terminology.
 - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.
 - For a librarian, it goes without saying that all purchases are arranged before they are added to the library.

- Changing requirements.

- Needs can change over time due to new stakeholders or changing business environments.

⇒ Ethnography:

An observational technique to understand how people actually work.

- Purpose:

To find out what user needs from a system by observing their everyday tasks.

- Process:

An ethnographer spends time with users, watching how they perform to identify system requirements.

- Benefits:

Provides deep insights into actual work practices that might not be captured through interviews or written descriptions.

- Challenges:

Requires the ethnographer to build trust with users and understand their work without interfering.

⇒ Requirements validation:

- Purpose:

Ensures that the requirements truly represent what the user needs.

• Checks:

Includes checking for validity (correctness), consistency (no conflicts), completeness (all needed functions included), realism (feasibility), and verifiability (testability).

• Importance:

→ Fixing requirements error is costly, so validation is crucial to avoid mistakes.

⇒ Requirements management:

The process of handling changes to requirements as the system is developed.

• **Need:** Requirements often change due to new business needs, technological changes, or feedback from users.

• **Process:** Involves tracking changes, assessing their impact, and updating documents accordingly.

⇒ Changing requirements:

• **Reasons:** Changes can come from new technology, new business priorities, regulatory changes, or new stakeholders.

• **Impact:** Changes need careful management to ensure they don't

disrupt the development process and lead to conflicts.

⇒ Requirement change management:

- Stages: Involves deciding if changes are needed, analyzing their impact, and implementing changes in a controlled way.

- Flexibility. Documents should be modular and easy to update to accommodate changes without extensive rewriting.

The 😊 End

